

Project (P)

Scalable Data Engineering
Solution Design & Implementation

Data Engineering I

Course Code: 1TD169

Spring Term 2026 / VT 26

Project Information

Total Points:	3 points (1 mandatory + 2 advanced)
Team Size:	5 students (4/6 with instructor approval)
Estimated Time:	max 50 hours per student
Checkpoint:	5-minute presentation (mandatory)
Final Submission:	Report + contribution statement + code repository
Due Date:	See course schedule on Studium

Contents

1 Overview & Learning Objectives	2
1.1 What You Will Learn	2
1.2 Project Philosophy	2
1.3 Team Composition	2
1.4 Suggested Timeline Overview	3
2 Project Requirements	4
2.1 Core Components	4
2.2 Dataset Selection	4
2.3 Analysis Objective	5
3 Implementation Guidelines	6
3.1 Resource Constraints & Architecture	6
3.2 Scalability Experiments (The Core Task)	6
3.3 Code Organization	6
3.4 Team Roles & Work Division	7
4 Checkpoint Presentation	8
4.1 Presentation Content	8
5 Final Report	9
5.1 Required Report Structure	9
5.2 Code Repository Requirements	10
6 Individual Contribution Statements	11
6.1 Content Requirements	11
6.2 Template (Optional)	12
7 Grading Rubric	13
7.1 1 Point (Pass) – The Mandatory Baseline	13
7.2 2 Points – Advanced Execution	13
7.3 3 Points – Exemplary Mastery	14
7.4 What counts as "Analysis"?	14
8 Teamwork Best Practices	15
8.1 Kickoff Meeting Agenda	15
8.2 Conflict Resolution	15
9 Technical Tips & Resources	16
9.1 Debugging Tips	16
9.2 Useful Resources	16
10 Submission Checklist	17
10.1 Before Final Submission	17
10.2 Submission Method	17
11 Frequently Asked Questions	19
12 Final Thoughts	20

1 Overview & Learning Objectives

The project provides hands-on experience with real-world data engineering challenges by designing and implementing a scalable data processing solution. Working in teams, you will select a dataset, architect a distributed system, and demonstrate its scalability through rigorous experimentation.

1.1 What You Will Learn

Through this project, you will:

- ✓ Apply course concepts (cloud, Hadoop, Spark) to a real problem
- ✓ Design horizontally and/or vertically scalable data pipelines
- ✓ Work collaboratively using Git and modern development practices
- ✓ Conduct systematic scalability experiments
- ✓ Communicate technical solutions through writing and presentations
- ✓ Make architectural trade-off decisions under constraints

1.2 Project Philosophy

Additional Info: Focus on Engineering, Not Analysis

The **primary goal** is demonstrating scalable data engineering solutions, not advanced data analysis. Your analysis objective may be **simple but realistic** (e.g., filtering, aggregation, feature extraction) to allow focus on:

- System architecture and tool selection
- Performance optimization and bottleneck analysis
- Scalability demonstration through experiments
- Code quality, reproducibility, and documentation

Example: Processing 10GB of Twitter data to extract tweet counts by country is sufficient. The engineering challenge is making it scale to 100GB or 1TB efficiently.

1.3 Team Composition

Standard team size: 5 students

Special circumstances: 4 or 6 students (requires instructor approval)

Important: Form Teams Early

Teams should ideally be finalized by 24th February on Studium. You may:

- Self-organize with classmates - select the first available empty group on Studium
- Or be assigned randomly by the instructors

Please get in touch with the head teacher (Prashant Singh) in case your group has members who have left the course, or do not participate.

1.4 Suggested Timeline Overview

Date	Milestone
Feb 19	Project Announcement & Introduction
Feb 24	<i>Recommended:</i> Teams formed & Dataset selected
Feb 25–Mar 8	Implementation & Scalability Experiments
Mar 9–10	Mid-Way Project Presentations
Mar 11–15	Wrap-up Experiments + Report writing
Mar 16	Final Deadline (Report + Contribution Statement)

💡 Quick Tip: Start Early

Successful projects allocate time for experimentation, failure, and iteration. Infrastructure setup alone may take a few days. Do budget time accordingly!

2 Project Requirements

2.1 Core Components

Every project must include:

1. **Dataset Selection:** Choose or find a publicly available dataset
2. **Analysis Objective:** Define a (possibly simple) preprocessing/analysis task
3. **Scalable Architecture:** Design and implement a distributed solution
4. **Scalability Experiments:** Demonstrate performance scaling
5. **Code Repository:** Well-organized Git repository with documentation
6. **Written Report:** Technical report (max 2500 words)
7. **Checkpoint Presentation:** 5-minute midpoint presentation
8. **Individual Contribution Statement:** Uploaded separately via Studium

2.2 Dataset Selection

Freedom of choice: Select any public dataset aligned with your interests.

Recommended sources:

- **Million Song Dataset:**¹ <http://millionsongdataset.com/>
Note: The full dataset is 280GB. We recommend starting with the 10K subset (~2GB) for development and scaling up to a larger subset (e.g., 10-15%) for the final run. Do not attempt to download the full 280GB.
- **Reddit Comments:**¹ <https://zenodo.org/records/1043504>
This massive collection of Reddit post-summary pairs is ideal for benchmarking distributed text processing pipelines (e.g., JSON parsing, tokenization, filtering) to demonstrate I/O and CPU scalability without requiring complex machine learning.
- **NYC Taxi Trips:** <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
Excellent for time-series and geospatial aggregation testing.
- **Awesome Public Datasets:** <https://github.com/awesomedata/awesome-public-datasets>
- **Kaggle Datasets:** <https://www.kaggle.com/datasets> (Look for "Big Data" tags)

Important: Dataset Approval Required

Before proceeding, get instructor approval for your dataset choice. Prepare answers to:

1. What is the total data size you intend to process?
2. What is the data format (CSV, JSON, Parquet, etc.)?
3. How will you ingest it into HDFS? (e.g., 'wget' to Master node, then 'hdfs put')
4. What is your planned analysis objective?

Post your proposal in your Studium group discussion forum for quick feedback.

¹A classic dataset for this course.

💡 Quick Tip: Dataset Size Guidelines for our Cluster

Minimum: 5 GB (Required to show basic distributed processing).

Target Range: 10–50 GB.

Why this range? Your cluster has limited RAM. 20 GB of raw JSON/CSV data often expands significantly in memory.

- If your data is < **5 GB**: You must duplicate it to simulate a larger load for scalability testing.
- If your data is > **50 GB**: You will likely hit performance bottlenecks. You must use efficient formats (e.g., Parquet^a) and careful partitioning.

^a<https://questdb.com/glossary/apache-parquet/>

📖 Additional Info: Data Format Considerations

Consider data formats impact on performance:

- **CSV/JSON**: Easy to work with, but slow parsing and no compression
- **Parquet/ORC^a**: Columnar formats, excellent compression, fast for analytics
- **Avro^a**: Row-based, good for write-heavy workloads

Part of your project may involve format conversion for optimization!

^aAdditional self-reading; not covered in the course

2.3 Analysis Objective

Define a **simple but realistic** data processing task. Examples:

Good objectives:

- Filter social media data by location and aggregate counts
- Extract features from log files for anomaly detection
- Compute user activity patterns from click streams
- Process sensor data to identify threshold violations
- Join datasets and compute summary statistics

Too complex (avoid):

- Training deep learning models
- Complex graph algorithms requiring global state
- Real-time streaming with sub-second latency requirements

🔍 Reuse Existing Code

You are encouraged to adapt existing analysis code from papers, GitHub, or tutorials. Citation is required. The engineering contribution (scalability) is what matters, not novel analysis algorithms.

3 Implementation Guidelines

3.1 Resource Constraints & Architecture

Your solution must be designed to run within the specific quotas of our course cluster.

⚠ Important: Strict Cluster Quotas per Group

To ensure fairness and stability for all 45+ groups, you must adhere to these limits:

- **Max VMs:** 4 Instances (Recommendation: 1 Master + 3 Workers).
- **Max Flavor:** `ssc.medium` (2 vCPU, 4GB RAM) or equivalent.
- **Max Storage:** 100 GB total attached volume capacity.
- **Floating IPs:** Max 1 (Attach to Master; use internal IPs for workers).
- **Replication:** Set HDFS replication factor to 2 (`dfs.replication=2`).

Recommended Technology Stack:

Component	Options
Storage	HDFS (Primary), Local FS (for staging)
Processing	Apache Spark (PySpark recommended)
Orchestration	Bash scripts, Makefiles
Monitoring	Spark History Server or <code>htop</code>

3.2 Scalability Experiments (The Core Task)

Since you cannot scale to infinity, your experiments should focus on **efficiency** and **bottleneck analysis** within the cluster limits.

📖 Additional Info: Scaling within Limits

Strong Scaling (Fixed Data, Increasing Nodes):

- **Scenario:** Run a job on a 5GB dataset.
- **Test A:** 1 Master + 1 Worker. Measure time.
- **Test B:** 1 Master + 2 Workers. Measure time.
- **Test C:** 1 Master + 3 Workers. Measure time.
- *Question:* Did you get a 3x speedup? If not, why? (Network overhead? Stragglers?)

Weak Scaling (Increasing Data, Increasing Nodes):

- **Scenario:** Keep the workload per node constant.
- **Test A:** 1 Worker processes 2GB.
- **Test B:** 2 Workers process 4GB.
- *Question:* Does the time stay constant?

Visualizing Results: Your report must include plots showing:

- Runtime vs. Number of Workers.
- Speedup Factor ($\text{Time}_1 / \text{Time}_N$) vs. Ideal Linear Speedup.

3.3 Code Organization

Your Git repository should be structured to allow anyone (including graders) to understand the workflow. Unless uploading a very small subset (in the megabytes), add the full dataset within

data/ to your .gitignore file immediately. Avoid pushing large datasets to GitHub.

Recommended Directory Structure:

```
project-repo/
|-- README.md           # Setup instructions & Architecture diagram
|-- data/               # Sample data (Do NOT commit large files!)
|-- infrastructure/     # Scripts to provision VMs (optional)
|-- src/
|   |-- etl_job.py      # Cleaning and ingestion logic
|   |-- analysis_job.py # Main Spark/MapReduce logic
|   +-- config.py       # Configuration (paths, constants)
|-- scripts/
|   |-- deploy.sh       # Helper to copy code to the cluster
|   +-- run_benchmark.sh # Script to run the 3 scaling tests automatically
|-- notebooks/          # Jupyter notebooks for initial data exploration
+-- docs/               # Contribution statements and design notes
```

3.4 Team Roles & Work Division

The project is designed for groups of 5. To ensure fair contribution, we suggest mapping members to these **Functional Areas**.

💡 Quick Tip: Suggested Roles for a 5-Person Team

1. **Infrastructure Engineer:** Owner of OpenStack, SSH keys, and network config. Ensures the cluster is healthy.
2. **Data Engineer (Ingestion):** Handles data download, cleaning, and HDFS formatting.
3. **Core Logic Developer:** Writes the main Spark/MapReduce algorithms and transformation logic.
4. **Test & QA Engineer:** Designs the benchmarks. runs the "1 vs 2 vs 3 worker" tests, collects logs.
5. **Analyst & Reporter:** Synthesizes the results, creates the plots, and leads the report writing.

Note: These are focus areas. Everyone must contribute to discussions, implementation, and writing the report. We understand that contribution levels and nature may vary slightly. However, everyone must understand and be able to explain the code.

4 Checkpoint Presentation

When: Mar 9th/10th (you only attend **one** session with your group – exact slot TBC later)

Format: 5-minute presentation + 2-minute Q&A

Attendance: Mandatory for all team members (talk to Prashant if you are not able to attend)

4.1 Presentation Content

Your checkpoint presentation should address:

1. Dataset & Context (1 min)

- What dataset did you choose? Why is it interesting/relevant?
- What is its size and format?

2. Analysis Objective (1 min)

- What specific task are you solving?
- Why does it need distributed processing?

3. Architecture Plan (2 min)

- What tools are you using (Spark, Hadoop, HDFS)?
- How are you structuring the system? Show an architecture diagram if available

4. Scalability Experiments Design (1 min)

- Horizontal or vertical scaling (or both)?
- What experiments will you run?
- What metrics will you measure?

5. Progress & Challenges (1 min)

- What have you completed so far?
- Any preliminary results?
- What are the main obstacles?

Quick Tip: Checkpoint Presentation Tips

- Use 3–5 slides maximum (please stay within 5 minutes)
- Include the architecture diagram
- Show preliminary results **only if available** (even failed experiments are instructive)
- Practice timing beforehand
- Ideally all members should speak/present (distribute sections) - discuss special circumstances with the head teacher (Prashant) if needed

Important: Missed Checkpoint

If you cannot attend due to illness/emergency:

1. Contact instructors immediately via email
2. Submit the supplementary assignment before the course ends

5 Final Report

Maximum length: 2500 words (excluding references, figures, code snippets)

Format: PDF, generated from LaTeX or Word (LaTeX recommended)

Submission: Upload to Studium as group submission

5.1 Required Report Structure

Title Page:

- Project title
- Course name and code
- Team members (full names and email addresses)
- Date

1. Background (~ 400 words)

Describe the scientific/application context of your dataset:

- What domain does it come from?
- Why is this dataset important/interesting?
- What analyses have others done with it? (cite 2–3 relevant papers/resources if applicable)
- What is your specific analysis objective?

2. Data Format & Characteristics (~ 300 words)

Describe the dataset technically:

- What format is it stored in (CSV, JSON, Parquet, etc.)?
- What is its schema/structure?
- Total size and number of records?
- Any data quality issues encountered?
- Why was this format chosen (or why did you convert it)?

3. System Architecture & Implementation (~ 800 words)

This is the **main section**. Describe:

- System design diagram (mandatory figure)
- Choice of tools (Spark vs. Hadoop vs. hybrid - justify!)
- Infrastructure configuration (VM setup, cluster topology)
- Data pipeline stages (ingestion → processing → output)
- Code structure and key algorithms
- Challenges faced and how you solved them

4. Scalability Experiments & Results (~ 700 words)

Present your experimental findings:

- Experimental design (what did you vary?)

- Results with plots (runtime, speedup, efficiency)
- Analysis of results (did scaling work as expected? why/why not?)
- Bottleneck identification (what limits scalability?)
- Comparison to theoretical expectations

5. Discussion & Conclusion (~ 300 words)

Reflect on the project:

- Was your architecture suitable for the problem?
- What worked well?
- What would you improve with more time/resources?
- What did you learn about distributed systems?

6. References

- Use consistent citation format (IEEE, ACM, or MLA)
- Minimum 4–5 references (dataset source, related work, tool documentation)

Quick Tip: Writing Strategy

Start early: Write Background and Data Format sections while still exploring.

Focus: Devote most word count to Sections 3–4 (architecture + experiments).

Figures: Figures don't count toward word limit - use them liberally as needed!

Proofread: Allocate time for team review and language check.

5.2 Code Repository Requirements

Submit a link to your Git repository (GitHub (preferred) /GitLab/Bitbucket) in the report.

Repository must include:

- Well-organized source code with comments
- README.md with setup and execution instructions
- Sample data or instructions to obtain it
- Scripts to reproduce experiments
- Requirements file (`requirements.txt` for Python or equivalent)

Repository visibility:

- Private repositories: Grant access to these 3 teachers (GitHub usernames: `prasi372`, `usamazf`, `zls0319`). Contact Prashant if your code is on a repo other than GitHub.

Important: Code Quality Also Matters

It is encouraged to consider:

- Code structure and readability
- Documentation quality
- Reproducibility (in principle, can we run your code?)

6 Individual Contribution Statements

Submission: Each team member submits **individually** via Studium (separate assignment link).

Format: PDF document, 1-page maximum.

6.1 Content Requirements

Your contribution statement must include:

1. **Full name and team members list**
2. **Your specific contributions**, structured by project phase:
 - **Planning:** What role did you play in design decisions?
 - **Implementation:** What code/components did you write?
 - **Experiments:** Did you design/run/analyze experiments?
 - **Writing:** Which report sections did you draft?
 - **Collaboration:** Code reviews, meetings, pair programming?

It is okay if you contributed to only some of the activities above.

3. **Acknowledgment of others' contributions**

Briefly describe what each teammate contributed.

4. **Estimated time contribution**

Your estimated hours spent vs. team average (e.g., “I spent approximately 55 hours, close to the 60-hour team average”).

5. **Signature and date**

Additional Info: Authorship Criteria

Your contribution statement should demonstrate that you meet scientific authorship criteria:

1. Substantial contributions to conception/design or implementation
2. Drafting or critical revision of written work
3. Final approval of submitted version
4. Accountability for all aspects of the work

Reference: https://academic.oup.com/gigascience/pages/authorship_guidelines

Important: Be Honest and Specific

Good example:

“I designed and implemented the Spark preprocessing pipeline (src/preprocessing.py, 300 lines), conducted horizontal scaling experiments (2-4 nodes), and drafted the System Architecture section. I also code-reviewed Alice’s analysis module and participated in planning meetings.”

Too vague (insufficient):

“I helped with the code and wrote some of the report.”

Consequences of misrepresentation: Individual grades may be adjusted based on contribution evidence (e.g., mismatched Git history, peer feedback, etc.).

6.2 Template (Optional)

```
INDIVIDUAL CONTRIBUTION STATEMENT
Data Engineering I (1TD169) - Project

Name: -----
Email: -----
Team Members: -----

MY CONTRIBUTIONS:

Planning & Design:
- [Describe your role in architecture decisions, tool selection, etc.]

Implementation:
- [List specific files/modules you wrote or significantly modified]
- [Mention lines of code or features implemented]

Experiments & Analysis:
- [Describe experiments you designed/conducted]
- [Mention data analysis or visualization you performed]

Documentation & Writing:
- [List report sections you drafted]
- [Mention README/code documentation you wrote]

Collaboration:
- [Code reviews, pair programming sessions, meeting organization]

Time Investment: Approximately ____ hours

TEAM MEMBERS' CONTRIBUTIONS:
- [Name]: [Brief description]
- [Name]: [Brief description]
...

I certify that this statement is accurate to the best of my knowledge.

Signature: -----
Date: -----
```

7 Grading Rubric

Grading Scale:

- **1 Point:** (Pass) - demonstrate either horizontal or vertical scalability + analyse results.
- **2 Points:** Breadth OR Depth: Horizontal/vertical + excellent analysis; OR Horizontal and vertical + basic analysis.
- **3 Points:** Breadth AND Depth: Horizontal and vertical scalability demonstration + comprehensive analysis.

7.1 1 Point (Pass) – The Mandatory Baseline

To pass in the project, you must demonstrate basic scalability.

✓ Success Criteria: Mandatory Checklist

Administrative & Functional:

- ✓ Pipeline is functional (Ingestion → Processing → Result).
- ✓ Checkpoint presentation attended (or supplementary assignment completed).
- ✓ Final report submitted with all required sections.
- ✓ Code repository submitted with clear README.md.
- ✓ Individual contribution statements submitted by all members (separately via Studium).

Scalability Requirement (Criteria for 1 point):

- ☑ You must perform **ONE** type of scalability experiment (Horizontal OR Vertical).
- ☑ You must include **plots** (Runtime vs. Nodes/Cores).
- ☑ You must include a **basic discussion** of the results (e.g., "It got faster," or "It didn't scale because...").

⚠ Important: Condition for U (Fail)

If you build the pipeline but **do not** run a scalability experiment (or do not include the plots), you cannot receive a passing grade.

7.2 2 Points – Advanced Execution

To achieve 2 points, you must build upon the passing requirements. You can qualify for this grade via **Path A (Breadth)** or **Path B (Depth)**.

🏆 Grading: Criteria for 2 points

Choose **ONE** of the following paths:

Path A: Breadth (Two Experiments)

- You demonstrate **BOTH** Horizontal (scaling nodes) **AND** Vertical (scaling cores) scalability.
- The analysis is basic but correct (plots are readable, trend is identified).

— OR —

Path B: Depth (One Experiment + Excellent Analysis)

- You demonstrate only **ONE** scaling type (e.g., Horizontal).
- The analysis is **excellent**: You identify bottlenecks (CPU vs. Network vs. I/O), explain outliers, and discuss Amdahl's Law or overhead penalties.

7.3 3 Points – Exemplary Mastery

To achieve the full 3 points, you must combine Breadth and Depth.

i Criteria for 3 points

You must fulfill ALL of the following:

1. **Breadth:** You demonstrate **BOTH** Horizontal AND Vertical scalability experiments.
2. **Depth:** Your analysis is **exemplary**.
 - *Comparison:* You explicitly compare Horizontal vs. Vertical scaling for your specific workload.
 - *Insight:* You explain *why* one method was better (e.g., "Vertical scaling hit a memory bandwidth wall, while Horizontal scaling was limited by shuffle").
3. **Report Quality:** High-quality writing, figures, and proper citations (4–5 relevant references).

7.4 What counts as "Analysis"?

To help you understand the difference between Basic, Excellent, and Exemplary analysis:

Level	Example Statement in Report
Basic (Pass)	"We increased nodes from 2 to 4. The time went down from 100s to 60s. The speedup was 1.6x."
Excellent	"The speedup was 1.6x (ideal is 2.0x). The loss in efficiency is likely due to the Shuffle phase. As seen in Figure 3, network traffic spiked during the sort operation, indicating we are network-bound."
Exemplary	"Comparing Horizontal vs. Vertical: Vertical scaling provided linear speedup (2.0x) because the dataset fits in the shared L3 cache. Horizontal scaling only provided 1.6x due to serialization overhead. Therefore, for this dataset size (< 10GB), scale-up is more cost-efficient than scale-out."

8 Teamwork Best Practices

8.1 Kickoff Meeting Agenda

Schedule a team meeting to align on:

1. Communication

- Primary channel (Slack, Discord, Teams, Email?)
- Meeting frequency
- Response time expectations

2. Ambition Level

- Aiming for 1 point (pass), 2 points, or 3 points?
- How much time can each member commit?

3. Git Workflow

- Repository setup (who creates it?)
- Branching strategy (feature branches? main protected?)
- Code review process (pull requests required?)

4. Initial Task Division

- Who explores which datasets?
- Who sets up initial infrastructure?
- Who drafts checkpoint presentation?

8.2 Conflict Resolution

If team issues arise (unequal work distribution, communication problems):

1. **First:** Evaluate if it can be addressed within the team (open conversation)
2. **Escalate:** Contact the head teacher (Prashant) if unresolved/you are not comfortable in solving it within yourselves.
3. **Last resort:** Individual grades may be adjusted based on contribution evidence

Important: Don't Wait Until the End

Problems compound over time. If someone is not contributing by the second week, address it immediately (via dialogue or informing the head teacher), please do not wait too long!

9 Technical Tips & Resources

9.1 Debugging Tips

Common issues:

- **Out of Memory:** Reduce `spark.executor.memory` or increase VM RAM
- **Slow jobs:** Check data locality, use `.cache()` for iterative algorithms
- **Cluster not found:** Verify master IP, check firewall rules
- **Data skew:** Check partition distribution, repartition if needed

 Quick Tip: Test Locally First

Before running on cluster:

1. Test with small data sample on single node
2. Debug logic errors locally
3. Then scale up to cluster

9.2 Useful Resources

Course materials:

- Lecture slides (Hadoop, Spark) on Studium
- Assignment 2 & 3 (reference for setup)
- Course Slack

External documentation:

- Spark Programming Guide: <https://spark.apache.org/docs/latest/>
- Hadoop Documentation: <https://hadoop.apache.org/docs/current/>
- OpenStack User Guide: <https://docs.openstack.org/user-guide/>

10 Submission Checklist

10.1 Before Final Submission

✓ Success Criteria: Final Checklist

Report:

- PDF format, max 2500 words (check word count!)
- All required sections included
- Figures numbered and referenced in text
- References formatted consistently
- Proofread for typos and clarity
- All team members listed on title page

Code Repository:

- Repository link included in report
- README.md with clear setup instructions
- All code committed and pushed
- No large data files in repo (add to .gitignore)
- Requirements file included
- Repository accessible to teachers

Individual Contributions:

- Each member submits contribution statement separately
- Statements are specific and honest
- Statements signed and dated

Experiments:

- Scalability experiments completed
- Results documented with plots
- Analysis of results included in report
- Scripts to reproduce experiments included in repo

10.2 Submission Method

Group submission (on Studium):

- Upload final report PDF
- One member submits on behalf of team
- Ensure all team members are associated with submission

Individual submission (separate Studium link):

- Each member uploads their contribution statement PDF
- Due same time as final report

⚠ Important: Late Submission Policy

Late submissions (without prior approval):

- Eligible for 1 point maximum (pass/fail only)
- No advanced points awarded
- Must still meet all mandatory requirements

If you need an extension due to illness/emergency, contact Prashant ASAP.

11 Frequently Asked Questions

Q: Can we use tools not covered in the course (e.g., Dask, Flink)?

A: You are allowed to, but **we strongly recommend using Spark or Hadoop**.

- *Reason 1:* The teaching team cannot provide debugging support for external tools.
- *Reason 2:* The course rubric focuses on specific distributed concepts best illustrated by the course tools.

Q: Our dataset is only 2 GB. Is that enough?

A: Ultimately the results should depict meaningful speedup as you scale, for which 2 GB might not be enough. As per the guidelines, we recommend a minimum of 5 GB. If your source is small, you may synthetically replicate it (e.g., duplicate the rows 10x) to demonstrate that your system handles scale.

Q: Can we use public cloud services (AWS, GCP) instead of SSC?

A: **Use at your own risk.**

- The university provides free resources (SSC).
- If you use AWS/GCP, you are personally responsible for all costs. We cannot assist with billing disputes or credit overruns.
- Your solution must still be reproducible for grading.

Q: What if our scalability experiments don't show speedup?

A: **That is a valid scientific result!** Often, adding nodes adds network overhead. If your curve is flat or goes down, you will still receive full points **IF** you analyze *why* (e.g., "The dataset was too small," or "The shuffle phase dominated the CPU gain").

Q: Can we change datasets after approval?

A: Only if strictly necessary (e.g., the data is corrupt). You must notify the instructor immediately. Changing datasets late in the project is the #1 cause of poor reports.

Q: Do we need to implement Machine Learning?

A: **No.** This is a Data Engineering course, not a Data Science course. Simple transformations (Filtering, Aggregation, Joins) are sufficient. We grade the *infrastructure scalability*, not the *algorithm complexity*.

Q: What if a team member stops contributing?

A: **Escalate immediately.** Do not wait too long. We can adjust individual grades based on the **Contribution Statement**, but we can best intervene if we know about the issue early.

Q: Can we use Generative AI (ChatGPT, Copilot)?

A: Yes. However:

- You must be able to explain every line of code if asked.
- "Copilot wrote a bug" is not a valid excuse for a failing pipeline.

Q: Do all team members need to present at the checkpoint?

A: Yes. The checkpoint is mandatory for the Pass (Grade 3) criteria. If you have a valid reason for absence (e.g., illness, other engagements), email the head teacher beforehand.

Q: Can we include code in the report?

A: Only short, critical snippets (max 10 lines) to explain logic. Do not paste whole files. Link to your Git repository instead.

12 Final Thoughts

This project is your opportunity to apply everything you’ve learned in a realistic scenario. Takeaways:

1. **Start early:** Infrastructure setup takes time
2. **Simple is better:** Complex analysis $\neq$ better grade; focus on engineering
3. **Document everything:** Future you (and graders) will thank you
4. **Iterate:** Expect to rebuild parts of your system
5. **Communicate:** Use team meetings, Slack, lab sessions
6. **Have fun:** Choose a dataset you find interesting!

Questions?

Post in Slack or contact instructors via email.

Good luck, and happy engineering!